Model Order Reduction techniques

Assignments - Part 2

Nicola R. Franco, Andrea Manzoni

Master's course in Mechanical Engineering

A.Y. 2024-2025

Students are required to pick (and elaborate upon) any of the three problems reported in the following pages.

All projects have a common skeleton. In particular, all case studies come with a suitable dataset containing a list of pre-computed FOM simulations. A showoff of the three case studies is available at the following link

https://colab.research.google.com/drive/1_kAQURJD0iv4ER2UosuTAClIPB0_yJUW?usp=sharing

The same notebook can be used to access and inspect the data.

Starting from there, students are requested to

- Implement, train and assess a data-driven ROM of choice (POD-NN, DL-ROM or, when appropriate, AE+SINDy);
- Exploit the ROM to answer a given question.

Students are expected to present their work by synthesizing the main steps in a suitable IPython notebook. To foster readability, technical pieces of code should be placed either outside of the notebook (i.e. within external ".py" files) or in suitable code sections that can be hidden/toggled by hand. To ensure a smooth online execution of the notebook, trained ROMs can be saved (once and for all) and later loaded on the fly.

The IPython notebook should report/contain:

i) a brief discussion about the design rationale, namely: why are we resorting to data-driven ROMs here? What is stopping us from using plain POD-Galerkin? Why did you pick a certain approach over the others[1]? How did you choose the latent/reduced dimension?;

ii) a ROM solver that can be queried in real-time, together with a short summary about its performances (computational cost, accuracy);

iii) any additional results concerning project-specific tasks.

[1] Or, in case you've tested multiple approaches and then simply selected the best one: can you explain why a certain ROM ended up working better than the others?

Leakage detection in a water reservoir (Navier-Stokes, stationary)

Artificial water reservoirs are commonly used to collect, store, and manage water for various purposes, including public supply of drinking water, irrigation, industrial use, hydroelectric power generation, flood control and more. For companies utilizing artificial water reservoirs, such as tanks and cisterns, proper maintenance is essential to prevent issues like water leaks and other potential mishaps.

Our purpose, here, is to come up with an efficient algorithm that can rapidly detect potential water leaks based on real-time measurements of the pressure field. To simplify, we consider a 2D section of a cylindrical cistern, consisting of a main core, $R := \{(x, y) \ : \ x^2 + y^2 \leq 0.5^2\} \subset \mathbb{R}^2$, surrounded by an emergency wall, $\Gamma = \{(x, y) \ : \ x^2 + y^2 = 1\}$. We refer to Figure 1 for a visual representation. In order to develop suitable safety measures, we are interested in studying the worst case scenario, that is, the unfortunate event in which water manages to break out of both the reservoir and the emergency wall. We use the steady Navier-Stokes equations to model the fluid flow, ideally capturing the scenario a few minutes after the break has occurred.

Let $\Omega = \{(x, y) \ : \ 0.5^2 < x^2 + y^2 < 1\}$ be the interspace between the reservoir and the emergency wall. Let $\boldsymbol{u} : \Omega \to \mathbb{R}^2$ and $p : \Omega \to \mathbb{R}$ be the velocity field and the pressure field of the fluid, respectively. Let $\mathbf{z}_1 \in \partial R$ and $\mathbf{z}_2 \in \Gamma$ be the break points on the two surfaces, respectively. Let $\rho_1, \rho_2 > 0$ describe the width of the two openings, respectively centered at $\mathbf{z}_1$ and $\mathbf{z}_2$. We model $\boldsymbol{u}$ and p as the solution to the following Navier-Stokes equations

$$\begin{cases} -\Delta \boldsymbol{u} + \boldsymbol{u} \cdot \nabla \boldsymbol{u} - \nabla p = 0 & \text{in } \Omega \\ \nabla \cdot \boldsymbol{u} = 0 & \text{in } \Omega \\ \boldsymbol{u} = 0 & \text{on } \partial\Omega, \end{cases} \tag{1}$$

with boundary conditions,

$$\begin{cases} \boldsymbol{u}(x, y) = \left[\frac{1}{4} + y^2 - \rho_1^2, \ 0.0\right]^\top & \forall (x, y) \in R \ : \ \|(x, y) - \mathbf{z}_1\| < \rho_1 \\ p(x, y) = 0 & \forall (x, y) \in \Gamma \ : \ \|(x, y) - \mathbf{z}_2\| < \rho_2 \\ \boldsymbol{u} = 0 & \text{elsewhere on } \ \partial\Omega. \end{cases} \tag{2}$$

modeling inflow, outflow and no-slip conditions, respectively. Given all the rotational symmetries enjoyed by Eqs. (1)-(2), we shall fix the position of the first break point, $\mathbf{z}_1 := [0.5, 0]$. Instead, we allow $\mathbf{z}_2, \rho_1, \rho_2$, to change as

$$0.05 \leq \rho_i \leq 0.1, \qquad \mathbf{z}_2 = [\cos\theta, \ \sin\theta]^\top \text{ with } \theta \in [0, \pi].$$

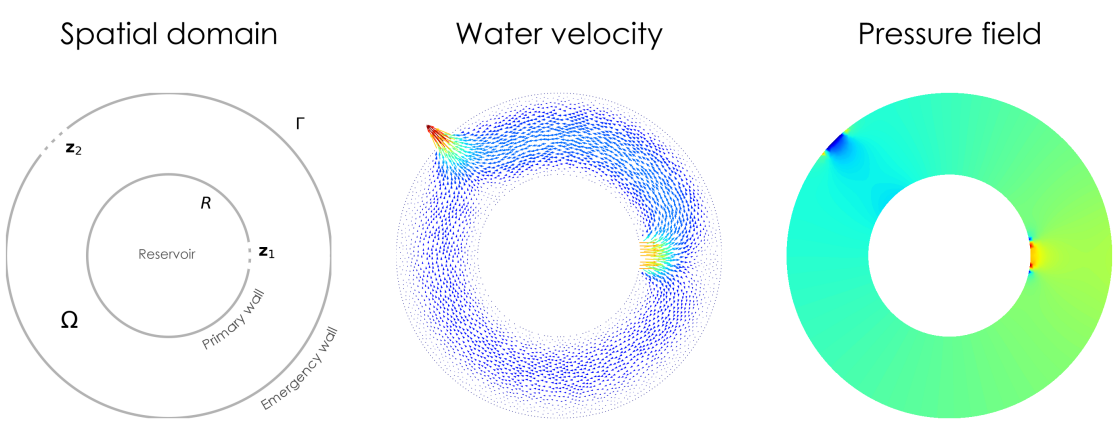

Figure 1. Schematic representation of the emergency interspace for the water-reservoir (left) and Finite Element simulation of water leaking out (center and right, respectively).

Tasks

Let $\boldsymbol{\mu} := [\theta, \rho_1, \rho_2]^\top$ be the problem parameters. You are given access[1] to a dataset of $N = 200$ pre-computed random simulations of the pressure field, obtained for different values of the problem parameters, $\{\boldsymbol{\mu}_i, p_{\boldsymbol{\mu}_i}\}_{i=1}^N$. The dataset was constructed by drawing random values for $\boldsymbol{\mu}$ (independently and uniformly in the parameter space) and then solving (1) using a Finite Element solver. Only the pressure fields were stored, with each PDE solution being represented as a discrete vector, $\mathbf{p}_{\boldsymbol{\mu}_i} \cong p_{\boldsymbol{\mu}_i}$, with $N_h = 3899$ degrees of freedom.

Additionally, you are also given access to a dataset of 8 sensor measurements coming from an unknown pressure field $p_* : \Omega \to \mathbb{R}$. These correspond to pressure values recorded at the outer boundary,

$$[s_1, \dots, s_k] = [p_*(x_1, y_1), \dots, p_*(x_8, y_8)]$$

with $(x_k, y_k) = (\cos k\frac{\pi}{4}, \sin k\frac{\pi}{4})$. Ideally, we would like to use the simulations provided and the sensor measurements to solve the inverse problem, that is, to recover the values θ, ρ_1, ρ_2 associated to $[s_1, \dots, s_k]^\top$. If we succeed, we will be able to estimate the position and the width of a water leak using as little as 8 sensor measurements.

i) To start, find the simulation in the dataset that best fits the measurements, that is

$$i_* = \operatorname*{argmin}_{i=1,\dots,200} \left\{ \sum_{k=1}^{8} |s_k - p_{\boldsymbol{\mu}_i}(x_k, y_j)|^2 \right\} = \operatorname*{argmin}_{i=1,\dots,200} \left\{ \sum_{k=1}^{8} |s_k - \mathsf{p}_{\boldsymbol{\mu}_i}^{(j_k)}|^2 \right\},$$

where $\mathbf{p}_{\boldsymbol{\mu}_i} = [\mathsf{p}_{\boldsymbol{\mu}_i}^{(1)}, \dots, \mathsf{p}_{\boldsymbol{\mu}_i}^{(N_h)}]$ and $j_1, \dots, j_8$ are suitable indices extracting the degrees of freedom associated to the sensors location. Which are the problem parameters associated to such simulation?

As a next step, we would to improve our estimate at point (a) by fitting a reduced order model over the data provided (rather than just using the dataset as an "atlas" for possible scenarios).

ii) Design and train a suitable ROM of your choice (POD-NN or DL-ROM) to approximate the map

$$\mathbb{R}^3 \ni \boldsymbol{\mu} \mapsto \mathbf{p}_{\boldsymbol{\mu}} \in \mathbb{R}^{3899}.$$

To this end, only use 150 simulations for training, keeping 50 for the testing phase. In the L^2 norm, what is the average relative error of your model over the test set? You may consider the ROM satisfactory if the latter is **below 12%**.

iii) Given any parameter vector $\boldsymbol{\mu}$, let $\tilde{\mathbf{p}}_{\boldsymbol{\mu}}$ denote the approximation of the pressure field obtained through the surrogate model. Leveraging the ROM, estimate the model parameters associated with the sensor measurements by solving the following least-square problem

$$\boldsymbol{\mu}_{\text{ltsq}} := \operatorname*{argmin}_{\boldsymbol{\mu} \in \Theta} \sum_{k=1}^{8} |s_k - \tilde{\mathsf{p}}_{\boldsymbol{\mu}_i}^{(j_k)}|^2,$$

with $\Theta = [0, \pi] \times [0.05, 0.1]^2$ being the parameter space.

[1] See the auxiliary notebook.

Damage propagation in a U-bolt (Fisher-KPP, time-dependent)

U-bolts are U-shaped fasteners with threaded ends that are commonly used to secure pipes, tubes, or rods to surfaces. Typically they are employed in structural applications, such as plumbing and automotive, to clamp cylindrical objects in place.

In this study, we aim to describe the propagation of micro-damages within U-bolts. With time, in fact, these bolts can accumulate small structural damages, typically caused by heavy loadings, stresses and material decay. Since we are not interested in sudden cracks or fractures, but in small failures that can propagate continuously, we model the spread of such damages using the Fisher-KPP equation.

Let $\Omega \subset \mathbb{R}^2$ be a domain representing a U-bolt —for the sake fo simplicity, we limit ourselves to a 2D framework. Let $u = u(\mathbf{x}, t)$ represent the structural damage at point $\mathbf{x}$ and time t. We set $0 \leq u \leq 1$, so that values close to 1 indicate a maximum damage, whereas values close to 0 indicate an undamaged or intact material state. We model the space-time field $u : \Omega \times [0, T] \to [0, 1]$ as the solution to

$$\begin{cases} \dfrac{\partial u}{\partial t} - D\Delta u = ru(1-u) & \text{in } \Omega \times (0, T] \\ \nabla u \cdot \mathbf{n} = 0 & \text{on } \partial\Omega \times (0, T] \\ u(\cdot, 0) = g_\delta & \text{in } \overline{\Omega}, \end{cases} \tag{3}$$

where $D > 0$ and $r > 0$ are the diffusion and reaction coefficient, respectively, whereas $\delta > 0$ is a parameter describing the location of the initial damage. Specifically,

$$g_\delta(x, y) := \begin{cases} 1 & |x - \delta| < 0.1 \text{ and } y \geq 4 \\ 0 & \text{otherwise.} \end{cases}$$

For the sake of our analysis, we parametrize the model coefficients as

$$D = 10^{\mu_1}, \qquad r = 10^{\mu_2 + 2}, \qquad \delta = \mu_3 + 1,$$

where $\boldsymbol{\mu} = [\mu_1, \mu_2, \mu_3]^\top$ varies in $[0, 1]^3$. The final time is set to $T = 0.02$.

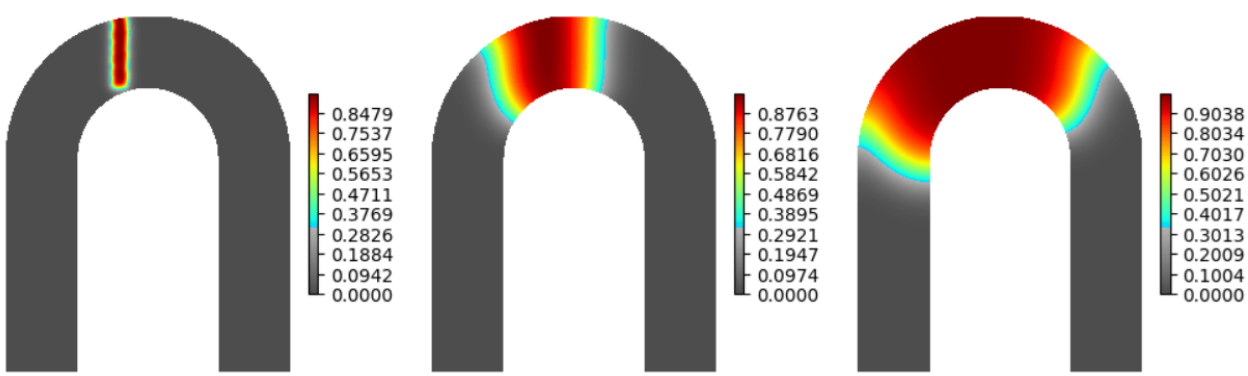

Figure 2. Three time stamps ($t = 0.0005, 0.01, 0.02$) of the first simulation in the dataset. Here, the PDE parameters are $\boldsymbol{\mu} = [0.549, 0.715, 0.603]$.

Tasks

You are given access[1] to a dataset of $N = 100$ pre-computed random simulations, obtained for different values of the problem parameters, $\{\boldsymbol{\mu}_i, u_{\boldsymbol{\mu}_i}\}_{i=1}^N$.The dataset was constructed by drawing random values for $\boldsymbol{\mu}$ (independently and uniformly in the parameter space) and then solving (3) using a Finite Element solver. In particular, each PDE solution is being represented as a matrix, $\mathbf{u}_{\boldsymbol{\mu}_i} \cong u_{\boldsymbol{\mu}_i}$, listing $N_t = 41$ time snapshots, each with $N_h = 6636$ degrees of freedom, equally spaced in time by $\Delta t = 0.0005$.

i) Split the data in two parts, keeping 75 trajectories for training and 25 for testing. Construct a reduced order model of your choice (POD-NN, DL-ROM, AE+SINDy) and evaluate its performances in terms of relative L^2 errors over the test set. You may consider the ROM satisfactory if you achieve an average relative error **below 10%** over the test set, that is

$$\frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \left[\frac{1}{N_t} \sum_{j=1}^{N_t} \frac{\|\mathbf{u}_{i,j}^{\text{test}} - \tilde{\mathbf{u}}_{i,j}^{\text{test}}\|_{L^2(\Omega)}}{\|\mathbf{u}_{i,j}^{\text{test}}\|_{L^2(\Omega)}} \right] < 0.1,$$

where $\mathbf{u}_{i,j}^{\text{test}}$ is the jth time stamp of the ith test simulation and $\tilde{\mathbf{u}}_{i,j}^{\text{test}}$ is the corresponding ROM approximation.

ii) How faster is the ROM considering that the simulation of a single trajectory via the FOM (41 time steps) requires ≈ 19.33 seconds?

We are interested in studying how the overall damage, averaged over time, is influenced by the problem parameters. To this end, we introduce the following functional

$$J(u) := \frac{1}{T} \int_0^T \int_\Omega u(\mathbf{x}, t) d\mathbf{x} dt,$$

modeling our quantity of interest.

iii) Focusing on the test set, compare (quantitatively) the values of J obtained by evaluating the functional over FOM and ROM solutions, respectively. Is the ROM reliable?

iv) Fix $D = \sqrt{10}$ and let $\delta \in \{1, 1.5, 2\}$. Leveraging the ROM, draw a plot showing how J changes as a function of $\log_{10}(r)$, where $100 \leq r \leq 1000$.

[1] See the auxiliary notebook.

Material design for a stunt training facility (linear elasticity, time-dependent)

A company is developing a high-performance flooring system to enhance safety in stunt training facilities. The design features a dual-layer structure: the bottom layer is stiffer, providing essential stability, while the top layer is softer, offering improved shock absorption to minimize the impact of falls.

As a part of the design process, the engineers working at the company are looking for the best compromise in terms of stability and impact absorption, trying to come up with the correct proportions between the two layers. To this, numerical simulations are being carried out.

Focusing on an ideal, isolated, portion of the floor, the engineers are modeling the behavior of the floor using the equations of linear elasticity. For simplicity, the analysis is conducted in two dimensions. Specifically, let $\Omega = (0,1)^2$ denote the portion of the floor under study. Let

$$\boldsymbol{f}(t) := \begin{cases} [0, -5\sin(2\pi t)]^\top & 0 \le t \le 0.5 \\ [0,0]^\top & t > 0.5 \end{cases}$$

be an external force representing the localized vertical impact exerted by an athlete's foot during contact with the floor. Set the final time to be $T = 1$. Let $\boldsymbol{u} : \Omega \times [0,T] \to \mathbb{R}^2$ be the vector-field describing the local displacement of the floor over time. We model $\boldsymbol{u}$ as the solution to

$$\begin{cases} \dfrac{\partial^2 \boldsymbol{u}}{\partial t^2} = \nabla \cdot \left[\lambda \nabla \cdot \boldsymbol{u} + \mu \left(\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^\top\right)\right] + \boldsymbol{g} & \text{in } \Omega \times (0,T] \\ \boldsymbol{u}(\cdot, 0) \equiv \boldsymbol{0} & \text{in } \Omega \\ \boldsymbol{u} \equiv \boldsymbol{0} & \text{on } \Gamma_{\text{bottom}} \times [0,T] \\ \lambda(\nabla \cdot \boldsymbol{u}) \cdot \boldsymbol{n} + \mu \left(\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^\top\right) \cdot \boldsymbol{n} \equiv \boldsymbol{f} & \text{on } \Gamma_{\text{top}} \times [0,T] \\ \lambda(\nabla \cdot \boldsymbol{u}) \cdot \boldsymbol{n} + \mu \left(\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^\top\right) \cdot \boldsymbol{n} \equiv \boldsymbol{0} & \text{on } \partial\Omega \setminus (\Gamma_{\text{bottom}} \cup \Gamma_{\text{top}}) \times [0,T] \end{cases} \tag{4}$$

where $\boldsymbol{g} = [0, -1]^\top$ is a fixed body force (caused by the floor own weight). The Lamé coefficients of the material are given as

$$\lambda(x,y) = \begin{cases} 10 & y \ge \delta \\ 50 & y < \delta \end{cases} \qquad \text{and} \qquad \mu(x,y) = \begin{cases} 10 & y \ge \delta \\ 50 & y < \delta, \end{cases}$$

where $\delta \in [0.5, 0.9]$ is a parameter describing the thickness of the two layers composing the floor.

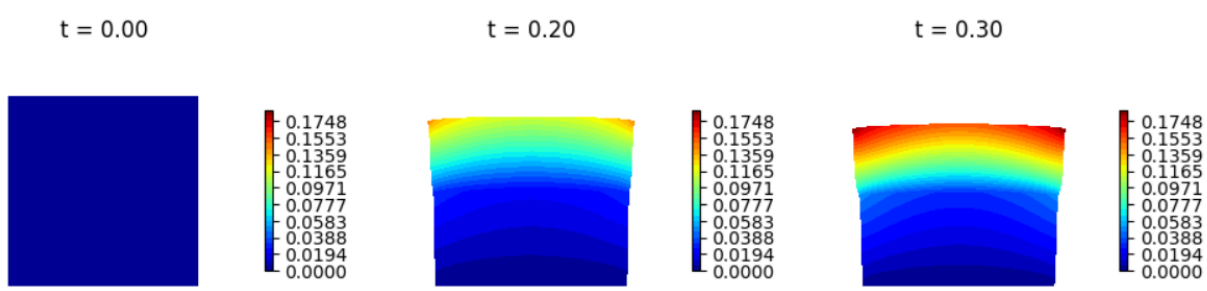

Figure 3. Three time steps of a FOM simulation, here obtained for $\delta = 0.5305$.

Tasks

Let $\boldsymbol{\mu} := \delta$ be the 1-dimensional parameter describing the system[1]. You are given access[2] to a dataset consisting of $N = 100$ pre-computed random simulations, $\{\boldsymbol{\mu}_i, \boldsymbol{u}_{\boldsymbol{\mu}_i}\}_{i=1}^N$, obtained for different values of the problem parameter. The dataset was constructed by drawing random values of $\boldsymbol{\mu}$, independently and uniformly in the parameter space, and by solving the corresponding equations of linear elasticity using Finite Element in space and finite differences in time. In particular, each PDE solution is being represented as a matrix, $\mathbf{u}_{\boldsymbol{\mu}_i} \cong u_{\boldsymbol{\mu}_i}$, listing $N_t = 51$ time snapshots equally spaced in time ($\Delta t = 0.02$), each with $N_h = 3362$ degrees of freedom.

i) Split the data in two blocks, keeping 75 simulations for training and 25 for testing. Choose a suitable approach between POD-NN, DL-ROM and AE+SINDy, and implement the corresponding reduced order model. Evaluate the performances of the ROM in terms of the following error metric (test average of relative space-time L^2 errors),

$$\frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \left[\frac{\sum_{j=1}^{N_t} \|\mathbf{u}_{i,j}^{\text{test}} - \tilde{\mathbf{u}}_{i,j}^{\text{test}}\|_{L^2(\Omega)}}{\sum_{j=1}^{N_t} \|\mathbf{u}_{i,j}^{\text{test}}\|_{L^2(\Omega)}} \right], \tag{5}$$

where $\mathbf{u}_{i,j}^{\text{test}}$ is the jth time stamp of the ith test simulation and $\tilde{\mathbf{u}}_{i,j}^{\text{test}}$ is the corresponding ROM approximation. You may consider your ROM to be satisfactory if the error in Eq. (5) is **below 2.5%**.

ii) Considering that the FOM requires ≈ 8.11 seconds to generate a single trajectory, quantify the computational speed-up provided by the ROM.

We are interested in studying how the thickness of the rigid layer, δ, affects the maximum deformation

$$J(\boldsymbol{u}) := \max_{(\mathbf{x},t)\in\Omega\times[0,T]} \|\boldsymbol{u}(\mathbf{x},t)\|.$$

iii) Leveraging the ROM, draw a plot of the map $\delta \mapsto J(\boldsymbol{u}_\delta)$. To foster interpretability, include also the values of J obtained by directly evaluating the functional over the training data and the test data (e.g., using "black dots" for training samples and "red dots" for test instances).

To gain additional insights, the company also wants to study the behavior of the maximum deformation over time, given by the time-dependent functional

$$J_t(\boldsymbol{u}) := \max_{\mathbf{x}\in\Omega} \|\boldsymbol{u}(\mathbf{x},t)\|.$$

iv) Pick 1001 equally spaced values of δ in [0.5, 0.9]. For each value of δ, use the ROM to draw a plot[3] of the map $t \mapsto J_t(\boldsymbol{u}_\delta)$. Does the time of maximum deformation change with δ? How?

[1] Even though we are in the scalar case, $p = 1$, we use the bold notation to avoid confusion with the shear modulus μ.
[2] See the auxiliary notebook.
[3] We can place all curves within the same plot, e.g. by letting the color fade from red to blue as you change the value of δ.